

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dvOretsky/fastapi-tutorial/blob/main/FAP18_Database/FAP18_Database.md

Практическое занятие №8: Подключение к БД и операции CRUD

Интеграция баз данных является важнейшим аспектом веб-приложений, поскольку она позволяет эффективно хранить, извлекать данные и управлять ими. **FastAPI** обеспечивает надежную поддержку интеграции **различных баз данных** в ваше приложение, позволяя вам беспрепятственно работать со структурированными данными. На данном занятии мы рассмотрим, **как подключить FastAPI к базам данных** и выполнять основные операции с базами данных.

Поддерживаемые базы данных в FastAPI

В FastAPI можно работать со всеми популярными базами данных, включая:

- **SQLite:** Облегченная бессерверная база данных, которая хранит данные в локальном файле.
- **PostgreSQL:** Мощная система управления реляционными базами данных с открытым исходным кодом, известная своей производительностью и масштабируемостью.
- **MySQL:** Популярная система реляционных баз данных, которая известна своей надёжностью.
- **MongoDB:** База данных NoSQL (нереляционная), которая хранит данные в гибких документах, подобных JSON.

Подключение FastAPI к базе данных

Чтобы подключить **FastAPI** к базе данных, вам необходимо выполнить следующие общие действия:

- **Шаг 1: Установите драйвер базы данных.** Сначала установите соответствующий драйвер базы данных для выбранной вами базы данных. Например, если вы планируете использовать PostgreSQL, установите библиотеку **asyncpg** для асинхронного доступа или **psycopg2** для синхронного доступа.
- **Шаг 2: Импортируйте драйвер базы данных.** В вашем приложении FastAPI импортируйте необходимый модуль драйвера базы данных.
- **Шаг 3: Настройте подключение к базе данных.** Настройте параметры подключения к базе данных, такие как URL-адрес базы данных, имя пользователя, пароль и другие

соответствующие конфигурации.

- **Шаг 4: Создайте сеанс работы с базой данных.** Создайте сеанс базы данных или пул подключений, который ваше приложение FastAPI может использовать для взаимодействия с базой данных.

Подключаем FastAPI к SQLite (без SQLAlchemy)

SQLite — это самая простая база данных, с которой можно работать. Она встроена в Python, и для нее не нужен отдельный сервер — данные просто хранятся в одном файле. Это делает **SQLite** отличным выбором для небольших проектов, тестов и локальной разработки.

Но вот в продакшене **SQLite** почти не используют. Почему?

- **Ограничение на одновременные подключения.** В отличие от PostgreSQL или MySQL, SQLite плохо справляется с большим числом одновременных запросов.
- **Отсутствие настоящего многопользовательского режима.** В крупных проектах база данных должна работать на отдельном сервере и поддерживать масштабирование.
- **Нет встроенной репликации.** В серьезных системах данные обычно копируются между несколькими серверами для отказоустойчивости.

Так что **SQLite** — это круто для старта, но если ваш проект вырастет, придется перейти на что-то мощнее.

Шаг 1: Устанавливаем зависимости

Нам понадобится библиотека `sqlite3`, которая уже встроена в Python, поэтому даже ничего ставить дополнительно не нужно.

Шаг 2: Настраиваем базу данных

Создадим файл `database.py`, который будет отвечать за подключение:

```
import sqlite3

DB_NAME = "database.sqlite"

def get_db_connection():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row # Это позволяет получать данные в виде словаря
    return conn
```

SQLite автоматически создаст файл `database.sqlite`, если его еще нет.

Шаг 3: Создаем таблицу

Теперь нам нужно создать таблицу. Запустите этот код один раз, чтобы база данных была готова:

```
import sqlite3

DB_NAME = "database.sqlite"

conn = sqlite3.connect(DB_NAME)
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS items (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
)
""")

conn.commit()
conn.close()
```

Этот код создаст таблицу `items`, где будут храниться записи.

Шаг 4: Подключаем базу к FastAPI

Теперь напишем основное приложение. Нам нужна ручка `POST /items`, которая будет добавлять данные в базу.

Создадим `main.py`:

```
from fastapi import FastAPI, Depends
from pydantic import BaseModel
import sqlite3
from database import get_db_connection

app = FastAPI()

class Item(BaseModel):
    name: str

@app.post("/items")
def create_item(item: Item):
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("INSERT INTO items (name) VALUES (?)", (item.name,))
    conn.commit()
    conn.close()

    return {"message": "Item added successfully!"}
```

Шаг 5: Тестируем API

Запустите сервер:

```
uvicorn main:app --reload
```

Затем отправьте POST-запрос, например, с помощью Postman или cURL:

```
curl -X 'POST' 'http://127.0.0.1:8000/items' \
  -H 'Content-Type: application/json' \
  -d '{"name": "Пример"}'
```

Если все работает правильно, вы получите:

```
{"message": "Item added successfully!"}
```

Сейчас это был простой вариант работы с базой без SQLAlchemy, чтобы сразу что-то добавить в БД. Как оказывается, работа с БД не так и страшна! Работу со всеми БД не показать в одном курсе, потому что их много, и часть из них платные, но принципы для всех одинаковы.

FastAPI + PostgreSQL: подключаемся без SQLAlchemy

Теперь у нас в деле **PostgreSQL** — серьёзная реляционная база, которая в отличие от SQLite не просто валяется файликом, а работает как полноценный сервер. Она умеет обрабатывать кучу запросов одновременно, поддерживает транзакции, индексы, связи между таблицами и вообще считается стандартом де-факто для большинства веб-приложений.

Короче, если у вас в продакшене SQLite — это как если бы сервер на Flask крутился на Raspberry Pi в углу офиса. Работает? Да. Надёжно? Ну такое себе...

Так что давайте разберёмся, как подключить FastAPI к PostgreSQL напрямую через **asyncpg**, без всяких ORM, чистым SQL.

Устанавливаем PostgreSQL

Если PostgreSQL у вас уже есть — красавчики, можно и не читать, наверное (раз она стоит, то зачем-то нужна, наверное уже с ней работали, и чему тогда учиться). Если нет, то вот вам краткая инструкция по установке:

- **Windows:** Скачиваем установщик с [официального сайта](#) и жмём «Далее, далее, готово».
- **macOS:** Просто кидаем в терминал: `brew install postgresql`
- **Linux (Ubuntu):** `sudo apt update` и `sudo apt install postgresql postgresql-contrib`.

Запускаем сервер:

```
sudo service postgresql start # Linux
pg_ctl -D /usr/local/var/postgres start # macOS (если ставили через brew)
```

Создаём базу и пользователя

После установки у вас уже есть админский пользователь **postgres**. Заходим под ним:

```
sudo -u postgres psql
```

И создаём пользователя и базу:

```
CREATE USER myuser WITH PASSWORD 'mypassword';
CREATE DATABASE mydatabase;
GRANT ALL PRIVILEGES ON DATABASE mydatabase TO myuser;
\q
```

На Винде работать с постгрей удобно через **PGAdmin**.

Подключаем Python к PostgreSQL

Ставим драйвер **asyncpg**, который позволит работать с базой асинхронно:

```
pip install asyncpg
```

Теперь создадим файл **database.py**, который будет управлять подключением. В отличие от SQLite, тут нельзя просто открыть файл — нужно поднимать соединение с сервером.

```
import asyncpg

# вбейте своего юзера, пароль и имя БД, если в команде выше изменяли
DATABASE_URL = "postgresql://myuser:mypassword@localhost/mydatabase"

async def get_db_connection():
    conn = await asyncpg.connect(DATABASE_URL)
    try:
        yield conn
    finally:
        await conn.close()
```

Каждый раз, когда нам нужно залезть в базу, мы открываем соединение, выполняем запрос и тут же его закрываем.

Создаём таблицу

PostgreSQL, в отличие от MongoDB, не создаёт таблицы «на лету», так что давайте создадим её вручную. Напишем скрипт `init_db.py`:

```
import asyncpg
import asyncio

DATABASE_URL = "postgresql://myuser:mypassword@localhost/mydatabase"

async def create_table():
    conn = await asyncpg.connect(DATABASE_URL)
    await conn.execute('''
        CREATE TABLE IF NOT EXISTS items (
            id SERIAL PRIMARY KEY,
            name TEXT NOT NULL
        )
    ''')
    await conn.close()

asyncio.run(create_table())
```

Запускаем один раз:

```
python init_db.py
```

Теперь в базе есть таблица `items`, в которую будем вносить данные.

Добавляем эндпоинт в FastAPI

Теперь самое интересное — сделаем ручку `POST /items`, которая будет добавлять элементы в базу. В файле `main.py`:

```
from fastapi import FastAPI, Depends
from pydantic import BaseModel
from database import get_db_connection
import asyncpg

app = FastAPI()

class Item(BaseModel):
    name: str

@app.post("/items")
async def create_item(item: Item, db: asyncpg.Connection =
    Depends(get_db_connection)):
    await db.execute('''
        INSERT INTO items(name) VALUES($1)
```

```
'''', item.name)
return {"message": "Item added successfully!"}
```

Теперь при каждом запросе **FastAPI** сам создаст соединение, выполнит запрос и закроет его.

Тестируем

Запускаем сервер:

```
uvicorn main:app --reload
```

И отправляем POST-запрос:

```
{
  "name": "Sample Item"
}
```

В ответ должно прийти:

```
{
  "message": "Item added successfully!"
}
```

Дальше по курсу будем разбираться с **MongoDB**, а ближе к финалу курса вернемся к работе с БД — прикрутим **SQLAlchemy**, чтобы не писать **SQL** вручную.

MongoDB + FastAPI

Ну что, завершаем этот блок и смотрим на **MongoDB**. В отличие от PostgreSQL и SQLite, это уже не реляционная база, а **NoSQL**. Здесь нет таблиц, колонок и строгой схемы — всё хранится в виде документов (по сути, JSON-объектов).

Mongo любят за гибкость, удобство работы с вложенными структурами и масштабируемость. Минус? Жёстких гарантий целостности данных тут нет, поэтому в финансовых сервисах её почти не встретишь, зато для логов, аналитики и гибких проектов — самое оно.

Теперь подключим FastAPI к MongoDB, без всяких сложных обвязок и ORM.

Устанавливаем MongoDB

Если Mongo уже стоит, идём дальше. Если нет, вот варианты:

1. **Windows / macOS / Linux:** Качаем с [официального сайта](#).
2. **Docker** (если не хотите ставить локально): `docker run -d --name mongodb -p 27017:27017 mongo`

3. Linux (Ubuntu): `sudo apt update`, `sudo apt install mongodb` и `sudo systemctl start mongodb`

Проверяем, что сервер работает:

```
mongo --eval "db.runCommand({ ping: 1 })"
```

Если ответ `{ "ok" : 1 }` — всё хорошо.

Ставим драйвер для Python

Для работы с MongoDB используем `motor` — асинхронный клиент для Python.

```
pip install motor
```

Настраиваем подключение

MongoDB по умолчанию работает на `mongodb://localhost:27017`. Подключение у нас будет происходить на лету, когда кто-то обратится к базе. Создадим `database.py`:

```
from motor.motor_asyncio import AsyncIOMotorClient

DATABASE_URL = "mongodb://localhost:27017"
DB_NAME = "mydatabase"

async def get_db():
    client = AsyncIOMotorClient(DATABASE_URL)
    db = client[DB_NAME]
    try:
        yield db
    finally:
        client.close()
```

Создаём коллекцию

В MongoDB не нужно заранее создавать коллекции (аналог таблиц). Они появятся автоматически при первом добавлении данных.

Добавляем эндпоинт в FastAPI

Теперь сделаем ручку `POST /items`, которая будет сохранять объект в базу. Создаём `main.py`:

```
from fastapi import FastAPI, Depends
from pydantic import BaseModel
from database import get_db
```



```
from motor.motor_asyncio import AsyncIOMotorDatabase

app = FastAPI()

class Item(BaseModel):
    name: str

@app.post("/items")
async def create_item(item: Item, db: AsyncIOMotorDatabase = Depends(get_db)):
    result = await db.items.insert_one(item.dict())
    return {"message": "Item added successfully!", "id": str(result.inserted_id)}
```

Mongo даже не требует описания схемы, но в реальных проектах лучше бы её всё-таки держать под контролем, например, через Pydantic.

Тестируем

Запускаем сервер:

```
uvicorn main:app --reload
```

Отправляем POST-запрос:

```
{
  "name": "Mongo Item"
}
```

В ответ получаем:

```
{
  "message": "Item added successfully!",
  "id": "65b72e8f2c4a3f8e3d96b1a7"
}
```

Mongo автоматически создаёт `_id`, который выглядит как длинная строка — это **ObjectId**, уникальный идентификатор записи.

Выполнение операций CRUD

Как только ваше приложение FastAPI подключено к базе данных, вы можете выполнять операции CRUD (создавать, читать, обновлять, удалять) с вашими данными.

- **Создать (CREATE):** Вставить новые записи данных в базу данных.

- **ЧИТАТЬ (READ):** Извлекать данные из базы данных на основе определенных критериев.
- **ОБНОВИТЬ (UPDATE):** Изменить существующие записи данных в базе данных.
- **УДАЛИТЬ (DELETE):** Удалить записи данных из базы данных.

Асинхронный доступ к базе данных

Встроенная поддержка **асинхронного программирования** в FastAPI позволяет вам использовать преимущества асинхронного доступа к базе данных. Используя синтаксис `async/await` с запросами к базе данных, ваше приложение может эффективно обрабатывать несколько запросов одновременно, что приводит к повышению производительности и масштабируемости.

Обратите внимание, что некоторые базы данных не позволяют асинхронно выполнять операции с целью поддержания стабильности данных. Поэтому существует мнение о **"неэффективности"** асинхронного программирования в Python (т.к. в конечном итоге приложение, которое может иметь сколько угодно потоков исполнения, все равно будет "ходить" в базу данных, которая все запросы будет обрабатывать по-очереди синхронно).

Примечание: При работе с базами данных в производственных приложениях не забывайте применять надлежащие меры безопасности, эффективно обрабатывать подключения к базе данных и оптимизировать запросы для обеспечения наилучшей производительности.

Введение в операции CRUD

Операции CRUD (создание, чтение, обновление, удаление) - это фундаментальные операции, используемые в управлении базами данных для взаимодействия с данными, хранящимися в базе данных.

Так как в FastAPI могут применяться различные базы данных, синтаксис которых может различаться, либо вы можете использовать более удобные формы взаимодействия с базами данных, такие как **ORM** (например PonyORM, рееее и др.), то примеры кода в данном уроке будут носить ознакомительный характер. Мы будем использовать **PostgreSQL**, так как в настоящее время это наиболее распространённая БД в российском продакшене (исходя из требований на hh.ru).

В связи с тем, что для разработки серверной части приложения в большинстве случаев требуется умение работать с БД, настоятельно рекомендуем вам пройти курсы по этой тематике (на степике имеются как платные, так и бесплатные).

Например начать знакомство с SQL можно с курса [SQL практикум](#).

Операция CREATE

Операция "Создать" (create) включает в себя добавление новых записей или вводимых данных в базу данных. В FastAPI вы можете создать новую запись, обработав запрос (например, POST-запрос) с соответствующими данными и сохранив его в базе данных.

Чтобы продемонстрировать операцию **"CREATE"** с использованием **PostgreSQL** в FastAPI, мы будем использовать библиотеку **"databases"** в качестве драйвера асинхронной базы данных и PostgreSQL в качестве базы данных.

Шаг 1: Установите необходимые библиотеки

Во-первых, убедитесь, что у вас установлены необходимые библиотеки. Вы можете установить адаптер базы PostgreSQL, выполнив следующую команду:

```
pip install databases[asyncpg]
```

Шаг 2: Создайте приложение FastAPI

Создайте новый файл с именем `main.py` и добавьте следующий код:

```
from fastapi import FastAPI, HTTPException
from contextlib import asynccontextmanager
from databases import Database
from pydantic import BaseModel

# URL для PostgreSQL (ЗАМЕНИТЕ user, password, localhost, dbname на свои
# реальные данные!)
DATABASE_URL = "postgresql://user:password@localhost/dbname"

# Главный объект для работы с базой данных – используется во всех запросах
database = Database(DATABASE_URL)

# Базовый класс для моделей пользователя – содержит общие поля
class UserBase(BaseModel):
    username: str
    email: str

# Модель для получения данных от клиента (валидация ввода)
# Наследует все поля от UserBase и может быть расширена дополнительными
# полями
# Пример: на входе мы можем запросить пароль, который не будем возвращать
# в ответе
class UserCreate(UserBase):
    """
    Входная модель для создания пользователя.
    В реальных проектах обычно содержит больше полей, чем выходная модель,
    например, пароль, подтверждение пароля или другие чувствительные
    данные.
    """
    pass # В текущей реализации поля совпадают с базовой моделью

# Модель для возврата данных клиенту (сериализация вывода)
# Наследует общие поля и добавляет технические данные из БД
# Важно: выходная модель часто содержит меньше полей, чем входная
class UserReturn(UserBase):
    """
    Выходная модель пользователя. Демонстрирует:
    – Добавление служебных полей (id из БД)
    – Исключение чувствительных данных (если бы они были)
    – Формат данных, безопасный для возврата клиенту
    """
```

```

    id: int # ID всегда присутствует после сохранения в БД

# Пример расширения моделей для учебных целей:
# class UserCreateWithPassword(UserCreate):
#     password: str
#     password_confirm: str

# class UserPrivateInfo(UserReturn):
#     created_at: datetime
#     last_login: datetime

# Управление подключением через lifespan (новый способ в FastAPI 0.95+)
@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    Контекстный менеджер для управления подключением к БД.
    Заменяет устаревшие @app.on_event("startup") и
    @app.on_event("shutdown")
    """
    # Устанавливаем соединение при старте приложения
    await database.connect()
    yield # Здесь работает приложение
    # Корректно закрываем подключение при завершении
    await database.disconnect()

app = FastAPI(lifespan=lifespan)

# Роут для создания пользователей с примерами валидации
@app.post("/users/", response_model=UserReturn)
async def create_user(user: UserCreate):
    """
    Создание пользователя с валидацией данных.

    Параметры:
    - user: данные согласно модели UserCreate

    Возвращает:
    - UserReturn с данными созданного пользователя и ID из БД

    Демонстрирует:
    - Разделение входных и выходных моделей
    - Автоматическую документацию в Swagger/OpenAPI
    - Обработку ошибок базы данных

    Пример использования транзакции:
    async with database.transaction():
        # несколько запросов в одной транзакции
        await database.execute(...)

    Дополнительно сам объект Database имеет свой асинхронный контекстный
    менеджер, то есть можно писать:
    async with Database(DATABASE_URL) as db:
        await db.execute(...)

```

Примеры выше полезны, если мы устанавливаем соединение не один раз при старте приложения,
а подключаемся к БД на каждый запрос (используем ресурсы по мере надобности, но чуть увеличиваем накладные расходы на создание соединения)

```

"""
# SQL-запрос с параметризацией (защита от SQL-инъекций)
query = """
    INSERT INTO users (username, email)
    VALUES (:username, :email)
    RETURNING id /* Получаем автоматически сгенерированный ID */
"""

try:
    # Пример использования транзакции (раскомментировать при
    # необходимости):
    # async with database.transaction():
    user_id = await database.execute(
        query=query,
        values=user.model_dump() # Автоматическая конвертация в
        словарь
    )

    # Комбинируем базовые поля с полученным ID
    return UserReturn(
        id=user_id,
        **user.model_dump(mode='json') # Сериализация для ответа
    )

except Exception as e:
    # В реальном проекте добавить логирование ошибки
    raise HTTPException(
        status_code=500,
        detail=f"Ошибка при создании пользователя: {str(e)}"
    )

```

Шаг 3: Настройка базы данных PostgreSQL

Убедитесь, что у вас запущен сервер PostgreSQL, и создайте новую базу данных и таблицу. Для этого примера давайте предположим, что у вас есть база данных с именем "mydatabase" и таблица с именем "users" со следующей структурой:

```

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL
);

```

Шаг 4: Запустите приложение

Чтобы запустить приложение FastAPI, используйте следующую команду:

```
uvicorn main:app --reload
```

Шаг 5: Протестируйте операцию "CREATE"

Используйте такой инструмент, как **"httpie"** или **Postman**, чтобы протестировать операцию "Создать", отправив POST-запрос с пользовательскими данными:

На httpie:

```
http POST http://localhost:8000/users/ username="john_doe"
email="john.doe@example.com"
```

Ответ должен быть таким:

```
{
  "username": "john_doe",
  "email": "john.doe@example.com",
  "id": 1
}
```

Пользователь был успешно добавлен в базу данных PostgreSQL с уникальным идентификатором.

Теперь у вас есть приложение FastAPI, которое демонстрирует операцию "Создать" с использованием PostgreSQL. Не забудьте настроить URL-адрес подключения к базе данных и структуру таблицы в соответствии с вашими конкретными настройками базы данных PostgreSQL и учетными данными. Также можете использовать PGAdmin для создания и просмотра содержимого вашей PostgreSQL базы данных. Например, можно использовать SQL-запрос прямо в PGAdmin:

```
SELECT * FROM users;
```

Что такое lifespan и как с ним работать

В FastAPI термин **lifespan** (или «жизненный цикл») относится к периоду работы всего приложения — от момента его запуска до завершения работы. Это концепция, позволяющая выполнять подготовительные действия перед обработкой запросов и корректно завершать работу, освобождая ресурсы.

Зачем нужен lifespan?

Раньше для выполнения кода на старте приложения и при его завершении использовались декораторы `@app.on_event("startup")` и `@app.on_event("shutdown")`. Однако, начиная с FastAPI 0.95, был введён новый способ управления жизненным циклом приложения — через параметр **lifespan** при создании экземпляра **FastAPI**. Это дает следующие преимущества:

- **Упрощение кода:** Вместо нескольких декораторов вы можете объединить логику инициализации и завершения в одной функции.
- **Управление ресурсами:** Это идеальное место для установки соединений с базами данных, настройке кэшей, логировании и других задач, требующих корректного закрытия ресурсов при завершении работы приложения.
- **Гибкость и читаемость:** Использование контекстного менеджера позволяет более явно показать, что происходит до и после обработки запросов.

Как работает lifespan?

Функция, отвечающая за **lifespan**, должна быть асинхронной и реализована как асинхронный контекстный менеджер. Для этого удобно использовать декоратор **@asynccontextmanager** из модуля **contextlib**. Такая функция должна иметь следующую структуру:

1. **Подготовительный этап (startup):** В начале функции вы выполняете все необходимые действия для подготовки приложения (например, устанавливаете соединение с базой данных).
2. **Рабочий этап:** С помощью оператора **yield** передаёте управление FastAPI для обработки запросов.
3. **Завершающий этап (shutdown):** После оператора **yield** пишется код, который будет выполнен при завершении работы приложения (например, закрытие соединения с базой данных).

Пример реализации lifespan-функции:

```
from fastapi import FastAPI
from contextlib import asynccontextmanager
from databases import Database

DATABASE_URL = "postgresql://user:password@localhost/dbname"
database = Database(DATABASE_URL)

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Подключаемся к базе данных при старте приложения
    await database.connect()
    yield # Здесь происходит обработка запросов
    # Закрываем соединение при завершении работы приложения
    await database.disconnect()

app = FastAPI(lifespan=lifespan)
```

Как использовать **lifespan** в вашем приложении?

1. Определение функции: Создайте асинхронную функцию, которая будет оборачивать логику инициализации и завершения. Важно, чтобы она использовала **yield** для передачи контроля.

2. Передача в FastAPI: При создании экземпляра FastAPI передайте функцию в параметр `lifespan`. Это укажет приложению, какие действия выполнять при запуске и завершении.

3. Работа с ресурсами: Внутри функции `lifespan` выполняйте подключение к ресурсам (например, открытие соединения с базой данных) до оператора `yield`, а после — освобождайте их (например, закрытие соединения).

Преимущества использования `lifespan`

- **Единая точка управления:** Вместо разделения логики на несколько функций, весь код, связанный с управлением ресурсами, находится в одном месте.
- **Повышенная надежность:** Гарантируется, что все ресурсы будут корректно освобождены при завершении работы приложения, что особенно важно для долгоживущих сервисов.
- **Интеграция с асинхронным кодом:** Позволяет легко работать с асинхронными операциями, такими как подключение к базам данных или выполнение других долгих операций.

Таким образом, использование `lifespan`-функции в FastAPI делает код более структурированным и упрощает управление жизненным циклом приложения, что особенно полезно при работе с внешними ресурсами.

Операция READ

Операция "Чтение" включает в себя извлечение данных из базы данных на основе определенных критериев, таких как идентификатор или параметры фильтрации. В FastAPI вы можете реализовать операции чтения, используя различные HTTP-методы, такие как запросы GET для извлечения данных из базы данных.

Можно расширить предыдущий код, добавив в него следующее:

```
from fastapi import FastAPI, HTTPException
from contextlib import asynccontextmanager
from databases import Database
from pydantic import BaseModel

# URL для PostgreSQL (ЗАМЕНИТЕ user, password, localhost, dbname на свои
данные!)
DATABASE_URL = "postgresql://user:password@localhost/dbname"

# Главный объект для работы с базой данных
database = Database(DATABASE_URL)

# Базовый класс для моделей пользователя
class UserBase(BaseModel):
    username: str
    email: str

# Модель для создания пользователя (входные данные)
class UserCreate(UserBase):
    """
```


Модель для получения данных от клиента.

В реальных проектах может содержать дополнительные поля, например, пароль, которые не возвращаются в ответе.

"""

Модель для возврата данных пользователя (выходные данные)

```
class UserReturn(UserBase):
```

"""

Модель для сериализации данных пользователя.

Включает технические поля из БД (id) и исключает чувствительные данные.

"""

id: int # ID всегда присутствует после сохранения в БД

Управление подключением к БД через lifespan

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
```

"""Управление жизненным циклом подключения к базе данных"""

```
    await database.connect()
```

```
    yield
```

```
    await database.disconnect()
```

```
app = FastAPI(lifespan=lifespan)
```

Эндпоинт для создания пользователей

```
@app.post("/users/", response_model=UserReturn)
```

```
async def create_user(user: UserCreate):
```

"""

Создание нового пользователя в базе данных.

Возвращает:

– UserReturn с данными созданного пользователя и ID из БД

Пример тела запроса:

```
{
    "username": "john_doe",
    "email": "john@example.com"
}
```

"""

```
query = """
```

```
    INSERT INTO users (username, email)
```

```
    VALUES (:username, :email)
```

```
    RETURNING id
```

"""

```
try:
```

```
    user_id = await database.execute(
```

```
        query=query,
```

```
        values=user.model_dump()
```

```
    )
```

```
    return UserReturn(
```

```
        id=user_id,
```

```
        **user.model_dump()
```

```
    )
```

```
except Exception as e:
```

```

        raise HTTPException(
            status_code=500,
            detail=f"Ошибка создания пользователя: {str(e)}"
        )

# Эндпоинт для получения пользователя по ID
@app.get("/users/{user_id}", response_model=UserReturn)
async def get_user(user_id: int):
    """
    Получение информации о пользователе по его ID.

    Параметры:
    - user_id: идентификатор пользователя в БД

    Возвращает:
    - Данные пользователя в формате UserReturn
    - 404 ошибку если пользователь не найден
    """
    query = """
        SELECT id, username, email
        FROM users
        WHERE id = :user_id
    """
    try:
        result = await database.fetch_one(
            query=query,
            values={"user_id": user_id}
        )
    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=f"Ошибка получения пользователя: {str(e)}"
        )

    if not result:
        raise HTTPException(
            status_code=404,
            detail="Пользователь с указанным ID не найден"
        )

    return UserReturn(
        id=result["id"],
        username=result["username"],
        email=result["email"]
    )

```

В приведенном выше коде мы добавили новый маршрут с именем `/users/{user_id}`, который принимает `user_id` в качестве параметра пути. Когда на этот маршрут отправляется запрос GET с действительным `user_id`, приложение запрашивает базу данных для пользователя с этим идентификатором и возвращает информацию пользователя в качестве ответа.

Например, если вы создаете пользователя с запросом POST (пример без httpie):

```
POST http://localhost:8000/users/

{
  "username": "vasya_pupkin",
  "email": "super_puper_mega_man@example.com"
}
```

То ответ будет:

```
{
  "username": "vasya_pupkin",
  "email": "super_puper_mega_man@example.com",
  "id": 2
}
```

Попробуйте отправить такой запрос (или можете поэкспериментировать с несуществующими `id`):

```
GET http://localhost:8000/users/1
```

Операция UPDATE

Операция "Обновление" включает в себя изменение существующих записей или вводов данных в базе данных. В FastAPI вы можете обрабатывать операции обновления, получая запрос с обновленными данными и обновляя соответствующую запись в базе данных.

Продолжаем заниматься модификацией нашего кода:

```
from fastapi import FastAPI, HTTPException
from contextlib import asynccontextmanager
from databases import Database
from pydantic import BaseModel

# URL для PostgreSQL (ЗАМЕНИТЕ user, password, localhost, dbname на свои
данные!)
DATABASE_URL = "postgresql://user:password@localhost/dbname"

# Главный объект для работы с базой данных
database = Database(DATABASE_URL)

# Базовый класс для моделей пользователя
class UserBase(BaseModel):
    username: str
    email: str

# Модель для создания пользователя (входные данные)
```

```

class UserCreate(UserBase):
    """
    Модель для получения данных от клиента.
    В реальных проектах может содержать дополнительные поля,
    например, пароль, которые не возвращаются в ответе.
    """

# Модель для возврата данных пользователя (выходные данные)
class UserReturn(UserBase):
    """
    Модель для сериализации данных пользователя.
    Включает технические поля из БД (id) и исключает чувствительные
    данные.
    """

    id: int # ID всегда присутствует после сохранения в БД

# Управление подключением к БД через lifespan
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Управление жизненным циклом подключения к базе данных"""
    await database.connect()
    yield
    await database.disconnect()

app = FastAPI(lifespan=lifespan)

# Эндпоинт для создания пользователей
@app.post("/users/", response_model=UserReturn)
async def create_user(user: UserCreate):
    """
    Создание нового пользователя в базе данных.

    Возвращает:
    - UserReturn с данными созданного пользователя и ID из БД

    Пример тела запроса:
    {
        "username": "john_doe",
        "email": "john@example.com"
    }
    """
    query = """
    INSERT INTO users (username, email)
    VALUES (:username, :email)
    RETURNING id
    """
    try:
        user_id = await database.execute(
            query=query,
            values=user.model_dump()
        )
        return UserReturn(
            id=user_id,
            **user.model_dump()
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Database error: {e}")

```

```
)
except Exception as e:
    raise HTTPException(
        status_code=500,
        detail=f"Ошибка создания пользователя: {str(e)}"
    )

# Эндпоинт для получения пользователя по ID
@app.get("/users/{user_id}", response_model=UserReturn)
async def get_user(user_id: int):
    """
    Получение информации о пользователе по его ID.

    Параметры:
    - user_id: идентификатор пользователя в БД

    Возвращает:
    - Данные пользователя в формате UserReturn
    - 404 ошибку если пользователь не найден
    """
    query = """
        SELECT id, username, email
        FROM users
        WHERE id = :user_id
    """
    try:
        result = await database.fetch_one(
            query=query,
            values={"user_id": user_id}
        )
    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=f"Ошибка получения пользователя: {str(e)}"
        )

    if not result:
        raise HTTPException(
            status_code=404,
            detail="Пользователь с указанным ID не найден"
        )

    return UserReturn(
        id=result["id"],
        username=result["username"],
        email=result["email"]
    )

# Роут для полного обновления информации о пользователе по ID
@app.put("/users/{user_id}", response_model=UserReturn)
async def update_user(user_id: int, user: UserCreate):
    """
    Полное обновление данных пользователя по ID (PUT-запрос).
    """
```

Параметры:

- user_id: ID пользователя в базе данных
- user: новые данные пользователя (все поля обязательны)

Возвращает:

- Обновленные данные пользователя в формате UserReturn
- 404 ошибку если пользователь не найден
- 500 ошибку при проблемах с базой данных

Пример запроса:

```
{
    "username": "new_username",
    "email": "new_email@example.com"
}
"""
# SQL-запрос с возвратом обновленных данных
query = """
    UPDATE users
    SET username = :username, email = :email
    WHERE id = :user_id
    RETURNING id
    """

values = {
    "user_id": user_id,
    "username": user.username,
    "email": user.email
}

try:
    # Выполняем запрос и получаем обновленные данные
    result = await database.execute(query=query, values=values)

    # Если запись не найдена
    if not result:
        raise HTTPException(
            status_code=404,
            detail="Пользователь с указанным ID не найден"
        )

    # Преобразуем результат запроса в модель UserReturn
    return UserReturn(**user.model_dump(), id=result,)

except HTTPException as he:
    # Пробрасываем HTTPException из проверки выше
    raise he

except Exception as e:
    # Обрабатываем другие ошибки базы данных
    raise HTTPException(
        status_code=500,
        detail=f"Ошибка обновления пользователя: {str(e)}"
    )
```

Теперь, если отправить PUT-запрос:

```
PUT http://localhost:8000/users/1

{
  "username": "updated_username",
  "email": "updated_email@example.com"
}
```

То ответ будет таким:

```
{
  "username": "updated_username",
  "email": "updated_email@example.com",
  "id": 1
}
```

Примечание: в практике принято для полного обновления ресурса использовать **PUT-метод**, а для частичного - **PATCH**.

Операция DELETE

Операция "Удалить" включает в себя удаление записей данных из базы данных. В FastAPI вы можете обрабатывать операции удаления, обрабатывая запрос с уникальным идентификатором (например, **ID**) и удаляя соответствующую запись из базы данных.

В целом как вы уже догадались, разницы особой не будет с предыдущей информацией, поэтому просто приведем демонстрационный код для HTTP-Delete запроса (интересует последний роут):

```
from fastapi import FastAPI, HTTPException
from contextlib import asynccontextmanager
from databases import Database
from pydantic import BaseModel

# URL для PostgreSQL (ЗАМЕНИТЕ user, password, localhost, dbname на свои
данные!)
DATABASE_URL = "postgresql://user:password@localhost/dbname"

# Главный объект для работы с базой данных
database = Database(DATABASE_URL)

# Базовый класс для моделей пользователя
class UserBase(BaseModel):
    username: str
    email: str

# Модель для создания пользователя (входные данные)
```

```

class UserCreate(UserBase):
    """
    Модель для получения данных от клиента.
    В реальных проектах может содержать дополнительные поля,
    например, пароль, которые не возвращаются в ответе.
    """

# Модель для возврата данных пользователя (выходные данные)
class UserReturn(UserBase):
    """
    Модель для сериализации данных пользователя.
    Включает технические поля из БД (id) и исключает чувствительные
    данные.
    """

    id: int # ID всегда присутствует после сохранения в БД

# Управление подключением к БД через lifespan
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Управление жизненным циклом подключения к базе данных"""
    await database.connect()
    yield
    await database.disconnect()

app = FastAPI(lifespan=lifespan)

# Эндпоинт для создания пользователей
@app.post("/users/", response_model=UserReturn)
async def create_user(user: UserCreate):
    """
    Создание нового пользователя в базе данных.

    Возвращает:
    - UserReturn с данными созданного пользователя и ID из БД

    Пример тела запроса:
    {
        "username": "john_doe",
        "email": "john@example.com"
    }
    """
    query = """
        INSERT INTO users (username, email)
        VALUES (:username, :email)
        RETURNING id
    """
    try:
        user_id = await database.execute(
            query=query,
            values=user.model_dump()
        )
        return UserReturn(
            id=user_id,
            **user.model_dump()
        )
    
```



```
)
except Exception as e:
    raise HTTPException(
        status_code=500,
        detail=f"Ошибка создания пользователя: {str(e)}"
    )

# Эндпоинт для получения пользователя по ID
@app.get("/users/{user_id}", response_model=UserReturn)
async def get_user(user_id: int):
    """
    Получение информации о пользователе по его ID.

    Параметры:
    - user_id: идентификатор пользователя в БД

    Возвращает:
    - Данные пользователя в формате UserReturn
    - 404 ошибку если пользователь не найден
    """
    query = """
        SELECT id, username, email
        FROM users
        WHERE id = :user_id
    """
    try:
        result = await database.fetch_one(
            query=query,
            values={"user_id": user_id}
        )
    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=f"Ошибка получения пользователя: {str(e)}"
        )

    if not result:
        raise HTTPException(
            status_code=404,
            detail="Пользователь с указанным ID не найден"
        )

    return UserReturn(
        id=result["id"],
        username=result["username"],
        email=result["email"]
    )

# Роут для полного обновления информации о пользователе по ID
@app.put("/users/{user_id}", response_model=UserReturn)
async def update_user(user_id: int, user: UserCreate):
    """
    Полное обновление данных пользователя по ID (PUT-запрос).
    """
```

Параметры:

- user_id: ID пользователя в базе данных
- user: новые данные пользователя (все поля обязательны)

Возвращает:

- Обновленные данные пользователя в формате UserReturn
- 404 ошибку если пользователь не найден
- 500 ошибку при проблемах с базой данных

Пример запроса:

```
{
    "username": "new_username",
    "email": "new_email@example.com"
}
"""
query = """
    UPDATE users
    SET username = :username, email = :email
    WHERE id = :user_id
    RETURNING id, username, email
    """

values = {
    "user_id": user_id,
    "username": user.username,
    "email": user.email
}

try:
    result = await database.fetch_one(query=query, values=values)

    if not result:
        raise HTTPException(
            status_code=404,
            detail="Пользователь с указанным ID не найден"
        )

    return UserReturn(**result)

except HTTPException as he:
    raise he

except Exception as e:
    raise HTTPException(
        status_code=500,
        detail=f"Ошибка обновления пользователя: {str(e)}"
    )

# Роут для удаления пользователя по ID
@app.delete("/users/{user_id}", response_model=dict)
async def delete_user(user_id: int):
    """
    Удаление пользователя из базы данных по ID.
```

```

Параметры:
- user_id: идентификатор пользователя для удаления

Возвращает:
- Сообщение об успешном удалении
- 404 ошибку если пользователь не найден
- 500 ошибку при проблемах с базой данных
"""
query = """
    DELETE FROM users
    WHERE id = :user_id
    RETURNING id
"""
try:
    # Пытаемся удалить запись и получить подтверждение
    deleted_id = await database.execute(
        query=query,
        values={"user_id": user_id}
    )

    if not deleted_id:
        raise HTTPException(
            status_code=404,
            detail="Пользователь с указанным ID не найден"
        )

    return {"message": "Пользователь успешно удален"}

except HTTPException as he:
    raise he

except Exception as e:
    raise HTTPException(
        status_code=500,
        detail=f"Ошибка удаления пользователя: {str(e)}"
    )

```

Реализация CRUD-операций в FastAPI

Чтобы реализовать операции **CRUD** в **FastAPI**, вам необходимо:

- **Определите модель данных**, которая представляет структуру ваших данных.
- **Создайте необходимые конечные точки API** (например, используя декораторы FastAPI) для каждой операции CRUD.
- **Используйте запросы к базе данных** (например, запросы на чистом SQL) для взаимодействия с базой данных и выполнения соответствующих операций.

Асинхронные операции CRUD

Встроенная поддержка асинхронного программирования в FastAPI позволяет выполнять асинхронные операции CRUD для повышения производительности. Используя асинхронные запросы

к базе данных (например, `async SQLAlchemy`, Tortoise ORM, `peewee-async` и тп), вы можете эффективно управлять несколькими операциями с базой данных одновременно.

FastAPI поддерживает различные **драйверы** асинхронных баз данных, например такие как `asyncpg` и `databases`.

Использование асинхронных операций с базой данных в FastAPI дает несколько преимуществ:

- **Повышенная производительность:** асинхронные запросы к базе данных позволяют приложению обрабатывать несколько запросов одновременно, сокращая общее время отклика.
- **Масштабируемость:** Асинхронное программирование обеспечивает лучшее использование ресурсов и масштабируемость, облегчая работу с высокими параллельными нагрузками.
- **Неблокирующее выполнение:** Асинхронные операции с базой данных предотвращают блокировку приложения во время ожидания ответа базы данных, обеспечивая лучшую оперативность реагирования.

Асинхронные операции с базой данных особенно полезны для обработки задач, связанных с вводом-выводом (I-O Bound), таких как **выборка данных из удаленной базы данных**.

Для выполнения асинхронных запросов к базе данных мы используем асинхронные функции и выражения ожиданий (**корутины и awaitable-объекты**). Асинхронные функции (корутины), которые взаимодействуют с базой данных, обозначаются с использованием синтаксиса `async def`. В рамках этих функций мы можем использовать ключевое слово `await`, чтобы приостановить выполнение в ожидании завершения запроса к базе данных.

Обработка ошибок

При реализации операций **CRUD** крайне важно корректно обрабатывать ошибки. **FastAPI** предоставляет надежные механизмы обработки ошибок, такие как создание соответствующих исключений **HTTP** (например, `404 Not found`), когда запрашиваемый ресурс не существует или операция сталкивается с проблемой.

Для сохранения целостности (консистентности) данных в вашей БД обязательно ознакомьтесь с "**хорошими практиками**" работы с БД. Например, если устанавливаете соединение с БД в каждом роутере каждый раз и используете ещё и синхронный подход, то обязательно делайте это через блок `try/except`. Не забудьте в блоке `finally` соединение закрыть.

Сегодня мы углубились в выполнение **CRUD-операций** с помощью **FastAPI** и узнали, как манипулировать данными в подключенной базе данных. Реализуя операции создания, чтения, обновления и удаления, вы можете создавать мощные веб-приложения, которые взаимодействуют с базами данных и обеспечивают бесперебойную функциональность управления данными. На следующем уроке мы рассмотрим миграцию баз данных с помощью **Alembic**, которая обеспечивает плавное обновление схемы базы данных и управление версиями.